

Frontend Developer Assignment UIDAI Sandbox

Candidate: Your Name

Stack: React 19, Vite 8, Redux Toolkit, Vitest, Service Worker (JS)

GitHub: <https://github.com/<your-username>/vc-wallet-uidai-sandbox>

PART 1 Credential Dashboard & State Management

- Responsive grid/list of mock credentials (name, issuer, type, issue date, status badge, masked identifier). Mock identifiers only, no real PII.
- Redux Toolkit: credentialsSlice holds items, status, error, with an async thunk calling GET /api/credentials.
- Explicit UI states: Loading (shimmer skeleton + aria-live), Error (message + Retry), Empty (no credentials), Success (credential list, count). Demo buttons switch Success/Empty/Error scenarios.
- Mock API: Vite dev/preview middleware serves /api/credentials with a 900ms delay (so the loading state is visible) and scenario params.
- Performance: React.memo on cards/list, stable selectors, no extra effects/re-renders. Semantic HTML, keyboard-focusable controls.

PART 2 SecureDataMask Component

- Reusable component <SecureDataMask value="123456789012" />; masked by default (XXXX-XXXX-9012). Tap to Reveal shows full value; button flips to Tap to Hide with aria-pressed.
- Auto re-mask after exactly 10s; manual hide masks instantly; timers cleared on hide/unmount/re-reveal. Revealed value stays only in local state; never logged, never in URLs/localStorage/aria attributes.
- Accessibility: focusable button, visible focus ring, aria-live status announces revealed/hidden, WCAG-sufficient contrast.
- Note: masking is a UI privacy control, NOT cryptographic protection.

PART 3 Offline PWA Support & Automated Testing

- Service worker (public/service-worker.js): GET /api/credentials handled network-first; latest OK response cached in Cache Storage; on network failure serve cached list; if none, return [] JSON fallback. Non-GET requests are never cached; old caches purged on activate.
- Installable PWA: manifest.webmanifest + SVG icon + SW registration in main.jsx. UI shows Online/Offline chip and banner.
- Security note: Cache Storage is unencrypted per-origin device storage demo caches mock data only; production needs TTLs, logout purge, TLS, disk encryption.
- Tests (Vitest + RTL, fake timers): masked by default; hidden initially; reveal on click; SR announcement; masked again at t=10000 (9999 still revealed); state reset; timer cleanup (vi.getTimerCount()===0); manual hide; safe unmount; no console leak. 8/8 pass.

HOW TO RUN

npm install

npm run dev # start dev server (mock API + loading delay)

npm test # run automated tests

npm run build # production build

npm run preview # preview build (service worker active)

ASSUMPTIONS & LIMITATIONS

- Mock data only; no real Aadhaar/UIDAI API; single page; no auth.
- Demo identifiers kept in Redux state for display; production should fetch identifiers on demand. SW cache has no TTL/encryption.

SCREENSHOTS (placeholders)

[Dashboard] [Loading] [Error] [Empty] [Masked] [Revealed]
[Offline banner]